KUMASI TECHNICAL UNIVERSITY

[Confirm/insert: Faculty name]

[Confirm/insert: Department name]

INDUSTRIAL ATTACHMENT PROJECT REPORT

ACCOLM CHECK

A web tool that screens how websites collect personal data against Ghana's Data Protection Act, 2012 (Act 843)

A solution to a problem I encountered during my industrial attachment with Accolm

Name	Ernest Abel Nanabanyin Mensah
Student Number	052341360055
Programme	BTech Computer Technology
Attachment	Accolm (remote), 11 May – 30 September 2026
Academic Supervisor	[Confirm/insert: name]
Work-Based Supervisor	[Confirm/insert: name]

OCTOBER 2026

ABSTRACT

During my remote industrial attachment with Accolm (11 May to 30 September 2026) I trained in HTML, CSS, Bootstrap, JavaScript and SQL with MySQL. I learnt that web forms are where personal data enters a system, that browser-side checks are not security, and that Ghana's Data Protection Act, 2012 (Act 843) sets eight principles for anyone who processes personal data. However, I had no easy way to check whether a real website met even the basics of the Act, and many small Ghanaian businesses have the same gap.

This report describes Accolm Check, a web application I built to close that gap. A user pastes a web address; the server fetches the page safely, finds the forms and fields that collect personal data (including Ghana Card, mobile money and digital address fields), runs fourteen checks, and maps each finding to a principle in section 17 of the Act with a plain-language explanation and a fix. The result is a score out of 100 and a shareable, printable report. The app uses HTML, CSS, Bootstrap 5 and vanilla JavaScript on the front end, Node.js with Express on the server, and SQLite or MySQL with prepared statements for scan history. It is protected against Server-Side Request Forgery, and 27 automated tests passed. The tool is an automated first-pass screening, not legal advice.

TABLE OF CONTENTS

1. INTRODUCTION AND PROBLEM STATEMENT 1
1.1 Background 1
1.2 The problem I faced 1
1.3 The solution 1
1.4 Objectives 2
2. BACKGROUND AND RELATED WORK 2
2.1 Data Protection Act, 2012 (Act 843) 2
2.2 Form security and the OWASP guidance 3
2.3 Server-Side Request Forgery 3
2.4 Existing tools 4
3. DESIGN 4
3.1 Architecture 4
3.2 How one scan works 5
3.3 Checks and scoring 5
3.4 Database design 5
3.5 Visual identity 6
4. IMPLEMENTATION 6
4.1 Project structure 6
4.2 Recognising personal data fields 6
4.3 A check mapped to a principle 7
4.4 Safe fetching (SSRF protection) 7
4.5 Prepared statements and server-side validation 7
4.6 Demo sites 8
5. SECURITY CONSIDERATIONS 8
6. TESTING AND RESULTS 9
6.1 Automated tests 9
6.2 End-to-end results 9
6.3 Screenshots 10
7. LIMITATIONS, FUTURE WORK AND CONCLUSION 11
7.1 Limitations 11
7.2 Future work 11
7.3 Conclusion 12
REFERENCES 13

1. INTRODUCTION AND PROBLEM STATEMENT

1.1 Background

I did my industrial attachment with Accolm as a remote intern from 11 May to 30 September 2026. Accolm is an information technology company whose main areas are secure managed file transfer, web application security, network security, and application development and integration (Accolm, n.d.-a). My training covered HTML, CSS, Bootstrap, JavaScript and SQL with MySQL. Most of my practice work was web forms: building them, laying them out responsively, checking inputs with JavaScript, and learning how the data would be stored in a database.

Two things I learnt during the attachment led to this project. The first came from Accolm's own web application security page. It explains that malicious code can be typed into a search or login field, and that a firewall gives no protection against this type of attack (Accolm, n.d.-h). This showed me that checking a form in the browser improves the user's experience but is not security. Every value has to be checked again on the server. The second was Ghana's Data Protection Act, 2012 (Act 843). A form is where personal data enters a system. Once a form asks for names, phone numbers or email addresses, the Act applies, including the eight principles in section 17 and the rule in section 19 that personal data may only be processed if it is necessary, relevant and not excessive (Republic of Ghana, 2012).

1.2 The problem I faced

During the attachment I understood, in theory, that forms and databases collect personal data and that Act 843 sets rules for this. In practice, I had no easy way to check whether a real website met even the basics of the Act. My own practice forms were never connected to a server or a database, so I never had to answer questions such as "Is this form sent securely?", "Does the page say why it wants my phone number?" or "Is there a privacy notice?". The only way to answer them was to read the Act and then inspect a page's source code by hand, which is slow and easy to get wrong.

As a freelance web designer in Accra, I see that small businesses have the same gap. Many put contact, booking and registration forms on their websites, but most owners have never read Act 843, and a lawyer or a security audit is beyond their budget. [Confirm/insert: one example from my own freelance work, without naming the client, of a form that asked for more personal data than it needed]

1.3 The solution

Accolm Check is a web application that turns this manual inspection into a quick, repeatable screening. A user pastes a website address. The server fetches the page safely, finds the forms and fields that collect personal data, runs fourteen checks and maps every finding to one of the eight principles in section 17. Each finding has a plain-language explanation and a fix that a business owner can hand to their web designer. The result is a score out of 100 and a shareable report that can be printed or saved as a PDF. The tagline of the project is "Know what your website knows about people." The name Accolm Check is a nod to Accolm, the company where I met this problem

during my attachment; it is my own student project, not an official Accolm product, and it does not claim to be endorsed by Accolm.

Accolm Check is a first-pass screening, not legal advice. It reads only what any visitor can see. It cannot see how data is stored, who can access it, or whether a business is on the Data Protection Register. A good score does not mean a site complies with the Act; a low score shows where to look first. This limit is stated on every result and report.

1.4 Objectives

- To detect, from a page's HTML, the forms and fields that collect personal data, including Ghana-specific fields such as Ghana Card, mobile money and GhanaPost digital address.
- To check each page against practical signs of the eight principles in section 17 of Act 843, and against section 19 (minimality).
- To explain every finding in plain English with a concrete fix.
- To store each scan in a database with prepared statements and server-side validation, and to provide a scan history and a shareable, printable report.
- To fetch other people's websites safely, so the tool itself cannot be used to attack the server's own network (Server-Side Request Forgery).
- To include offline demo sites and automated tests, so the project can be demonstrated and checked without internet.
- To build it only with the technologies I trained in at Accolm (HTML, CSS, Bootstrap, JavaScript and SQL), plus a small Node.js server.

2. BACKGROUND AND RELATED WORK

2.1 Data Protection Act, 2012 (Act 843)

Act 843 establishes the Data Protection Commission and regulates how personal data is obtained, held, used and disclosed (Republic of Ghana, 2012). The sections the tool relies on are summarised in Table 2.1. I read each one in the Act text on GhaLII before using it.

Section	What it says (summary)	How Accolm Check uses it
s.17	Anyone who processes data must apply eight principles: accountability, lawfulness of processing, specification of purpose, compatibility of further processing, quality of information, openness, data security safeguards and data subject participation.	Every check is mapped to one principle. The report shows all eight.
s.19	Personal data may only be processed if the purpose is necessary, relevant and not excessive.	Minimality check: too many personal fields, or Ghana Card or date of birth in a simple enquiry form.
s.20	Personal data should not be processed without prior consent, unless another listed reason applies, such as a contract.	Consent checkbox check. A missing box is a warning, not a failure, because other lawful reasons exist.
s.23	The data subject must be made aware of the purpose of collection.	Purpose statement near each form.
s.26	Data must be complete, accurate, up to date and not misleading.	Email and phone fields use matching input types.
s.27	Data controllers must register with the Commission and make people aware of, among other things, the name and address of the person collecting the data.	Contact details check and the Data Protection Commission mention.
s.28	Data controllers must secure personal data with appropriate technical and organisational measures.	HTTPS, form transport, security headers and mixed content.
s.32 and s.33	Data subjects may request access to and correction or deletion of their data.	Rights section in the privacy notice.
s.37	Special personal data (for example religion, health, ethnic origin) is restricted.	A special-data field is a failure in the minimality check.
s.40	Personal data may not be used for direct marketing without prior written consent.	Marketing checkboxes must be separate and unticked.

Table 2.1: Sections of Act 843 used by the tool (Republic of Ghana, 2012)

2.2 Form security and the OWASP guidance

The Open Worldwide Application Security Project (OWASP) publishes free guidance that matches what Accolm's security page warns about. The input validation cheat sheet says validation must happen on the server, because anything done in the browser can be bypassed (OWASP Foundation, n.d.-b). The SQL injection prevention cheat sheet recommends prepared statements with parameters as the main defence (OWASP Foundation, n.d.-c). The OWASP Secure Headers Project describes response headers, such as Strict-Transport-Security and Content-Security-Policy, that tell the browser to protect the page (OWASP Foundation, n.d.-d). These became both the rules the scanner checks and the rules I followed when I built the scanner.

2.3 Server-Side Request Forgery

A tool that fetches any address a visitor types is risky. An attacker could ask it to fetch http://127.0.0.1 or the cloud metadata address 169.254.169.254 and read private data from inside

the server's own network. This is called Server-Side Request Forgery (SSRF), and it is listed in the OWASP Top 10 (OWASP Foundation, 2021). The OWASP SSRF prevention cheat sheet recommends allowing only the expected protocols, refusing private and internal IP addresses, checking the address that is actually connected to, and not following redirects blindly (OWASP Foundation, n.d.-a). The reserved IP ranges come from the IETF special-purpose address registry (Cotton et al., 2013).

2.4 Existing tools

General website scanners, such as security header checkers and cookie scanners, already exist, but the ones I know of test against international standards or the European GDPR. They do not recognise Ghana Card or mobile money fields, and they do not explain results using the principles of Act 843 in plain English for small Ghanaian businesses. That gap is what makes this project useful in Ghana. [Confirm/insert: any Ghana-specific tools I found when I searched, and how Accolm Check differs from them]

3. DESIGN

3.1 Architecture

The system has four parts (Figure 3.1). The front end is plain HTML, CSS, Bootstrap 5 and vanilla JavaScript. An Express server on Node.js receives scan requests, validates them and applies rate limiting. Behind it, a safe fetcher (ssrf.js) gets the page, and an analyser (scanner.js) runs the checks. Results are saved in SQLite by default, or in MySQL when the DB_DRIVER setting is mysql. I followed the stages of the software development life cycle on a small scale: requirements, design, implementation and testing (Sommerville, 2016). The layout uses Bootstrap's grid (Bootstrap, n.d.) and the server uses the Express framework (OpenJS Foundation, n.d.). I chose Node.js because the analyser is JavaScript, the same language I trained in, and the whole app runs with one npm start, with no web server or PHP set-up.

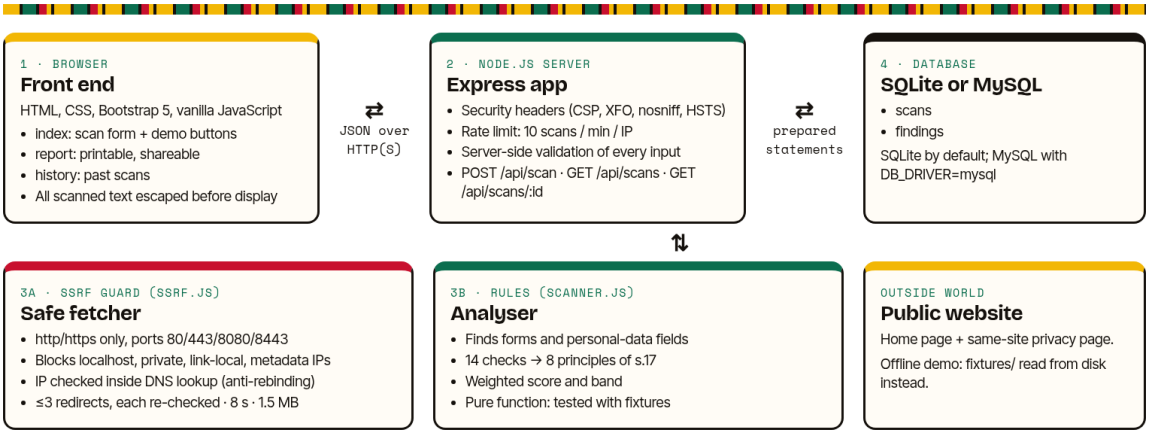


Figure 3.1: Architecture of Accolm Check

3.2 How one scan works

- The browser sends the address to POST /api/scan as JSON.
- The server validates the address. It must use http or https, a standard port and a public host name, with no username or password.
- The safe fetcher resolves the host name and refuses any private or internal IP address. It then downloads the page with an 8-second time limit and a 1.5 MB size limit, following at most three redirects and checking each one again.
- If the page links to a privacy notice on the same site, that page is fetched in the same safe way.
- The analyser reads the HTML with the cheerio library, finds the forms and classifies each field, runs the checks and calculates the score.
- The result is saved in the database and returned to the browser, which shows it and offers the shareable report link.

3.3 Checks and scoring

Each check has a weight from 1 to 3 according to its risk to people. A pass earns the full weight, a "check" (warning) earns half and a "fix now" earns nothing. The score is the earned weight divided by the total weight of the checks that apply, shown out of 100. Checks that do not apply are left out; for example, form checks are skipped on a page with no forms. A score of 80 or more is a "Strong start", 50 to 79 "Needs work", and below 50 "At risk". I chose these words on purpose: none of them says "compliant".

3.4 Database design

The database has two tables (Figure 3.2). The scans table holds one row per scan, with a random 12-character public_id used in share links, so report numbers cannot be guessed by counting. The findings table holds one row per check and links to its scan with a foreign key. This makes it possible to ask questions such as "which check fails most often?" with simple SQL. The full result is also kept as JSON so that a report can be redrawn exactly. No personal data from scanned sites is stored, only addresses, field categories and findings.

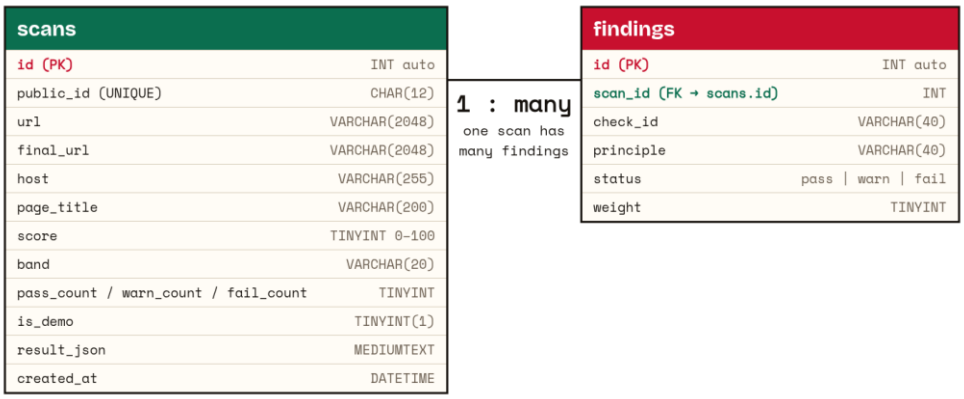


Figure 3.2: Database schema (MySQL types shown; the SQLite schema has the same columns)

3.5 Visual identity

I did not want the tool to look like a generic template. The pages use a warm paper background, near-black for buttons and actions, and a gold highlighter mark for anything the scanner flags, as if someone had marked up the form with a pen. Green, amber and red are kept for results only, and each status also has its own word and shape (a circle for Pass, a diamond for Check and a square for Fix now), so the meaning never depends on colour alone. Kente cloth stays as the signature: it forms the logo, a thin woven line at the top of every page, and a loading bar that moves like a loom while a scan runs. Headlines use the Bricolage Grotesque typeface, body text uses Inter Tight, and labels and evidence use Space Mono, so that technical evidence looks like evidence. The score is drawn as a ring out of 100, with a bar that shows how many checks need fixing, need checking or passed. The home page shows a sample quote form with its risky fields highlighted, then tells a short story in three steps: find the doors (forms), check the locks (security), and explain in plain words. The pages follow the visitor's light or dark setting, work on a phone, and print as a clean A4 report. The fonts and Bootstrap are served from the app itself, so it works offline.

4. IMPLEMENTATION

4.1 Project structure

The code is in the folder accolm-check. server.js starts the app. The src folder holds app.js (routes, headers, rate limit), ssrf.js (URL validation and safe fetching), scanner.js (the checks), act843.js (the principles), scan.js, demo.js and db.js. The public folder holds the three pages (home, report and history), a page-not-found page, and their CSS and JavaScript. The fixtures folder holds the two demo sites, and the test folder holds the automated tests.

4.2 Recognising personal data fields

For every input, select and textarea, the scanner joins the field's name, id, placeholder, aria-label and label text, and tests the combined text against an ordered list of rules (Listing 4.1). The order matters: "email address" must count as an email, not a postal address, so the email rule comes first. Ghana-specific patterns include "Ghana Card", the GHA- number format, "MoMo" and "GhanaPost".

```
const FIELD_RULES = [
  { key: 'email', test: (f) => f.type === 'email' || /e-?mail/.test(f.text) },
  { key: 'ghana_card', sensitive: true,
    test: (f) => /ghana[\s_-]?card|national[\s_-]?id|\bnia\b|gha-\d/.test(f.text) },
  { key: 'dob', sensitive: true, test: (f) => /date[\s_-]?of[\s_-
]?birth|\bdob\b/.test(f.text) },
  { key: 'momo', sensitive: true, test: (f) => /\bmomo\b|mobile[\s_-]?money/.test(f.text) },
  { key: 'special', special: true, test: (f) =>
/religio|ethnic|tribe|\bhealth\b|medical/.test(f.text) },
  { key: 'phone', test: (f) => f.type === 'tel' || /phone|mobile|whatsapp/.test(f.text) },
  { key: 'address', test: (f) => /address|street|digital[\s_-]?addr|ghanapost/.test(f.text)
},
  // ... name, gender, payment card, password
];
const classifyField = (f) => FIELD_RULES.find((r) => r.test(f)) || null;
```

Listing 4.1: Field rules in scanner.js (shortened)

4.3 A check mapped to a principle

Every check returns the same structure: an id, a title, the principle, the sections, a weight, a status, the evidence found, a plain explanation and a fix. Listing 4.2 shows the minimality check for section 19.

```
const special = form.personal.some((p) => p.special);
if (form.personal.length > 6) issues.push(`${form.id} asks for ${form.personal.length}
personal fields`);
if (looksLikeSimpleEnquiry && sensitiveFields.length) issues.push(`${form.id} asks for Ghana
Card or date of birth`);
add({
  id: 'minimality', title: 'Only necessary data is collected',
  principle: 'purpose', sections: ['s.19 (minimality)'], weight: 3,
  status: special ? 'fail' : issues.length ? 'warn' : 'pass',
  evidence: issues.join('; '),
  fix: 'For each field ask: "What breaks if we remove this?" ...',
});
```

Listing 4.2: The minimality check (simplified)

4.4 Safe fetching (SSRF protection)

The most important security code is the DNS lookup in ssrf.js (Listing 4.3). Node.js lets the app supply its own lookup function for each connection. Checking the IP address inside this function, rather than before the request, means the address that was checked is the address the socket connects to. This stops a "DNS rebinding" trick, where a host name answers with a public address the first time and a private one the second time. If a host name has several addresses and any one of them is private, the whole request is refused.

```
function makeSafeLookup(resolver = dns.lookup) {
  return function safeLookup(hostname, options, callback) {
    resolver(hostname, { ...options, all: true }, (err, addresses) => {
      if (err) return callback(err);
      if (addresses.length === 0 || addresses.some((a) => isBlockedIp(a.address))) {
        return callback(new UnsafeUrlError(`${hostname} points to a private or internal
address`));
      }
      if (options.all) return callback(null, addresses);
      return callback(null, addresses[0].address, addresses[0].family);
    });
  };
}
```

Listing 4.3: The IP check runs inside the socket's own DNS lookup

4.5 Prepared statements and server-side validation

All database work goes through db.js. Every query uses ? placeholders, so a value typed by a user is never pasted into SQL text (Listing 4.4). The same code runs on SQLite with better-sqlite3 and on MySQL with mysql2, and a scan and its findings are saved in one transaction. Before any value reaches the database, the server checks it again: the URL with validateUrl(), the demo name against a fixed list, and the report id against the pattern of exactly 12 letters, digits, dashes or underscores. JSON bodies larger than 4 KB are refused.

```
const [res] = await conn.execute(
  `INSERT INTO scans (public_id, url, final_url, host, page_title, score, band,
                      pass_count, warn_count, fail_count, is_demo, result_json)
   VALUES (?,?,?,?,?,?,?,?,?,?,?,?)`,
  [pid, r.url, r.finalUrl, host, r.title, r.score, r.band.key,
   r.counts.pass, r.counts.warn, r.counts.fail, demo ? 1 : 0, JSON.stringify(r)]);

app.get('/api/scans/:id', async (req, res) => {
  if (!/^[A-Za-z0-9_-]{12}$/.test(req.params.id)) return res.status(400).json({ error:
'Invalid report id.' });
  ...
```

Listing 4.4: A prepared statement (MySQL version) and server-side validation of a report id

4.6 Demo sites

The fixtures folder holds two fictional businesses. "Adwoa's Kitchen Catering" is deliberately non-compliant. It runs over plain HTTP, and its quote form uses GET and asks for a Ghana Card number, date of birth, religion and a MoMo number. The form also has a pre-ticked SMS offers box, Google Analytics and the Meta Pixel with no cookie notice, and no privacy notice or contact details. "Sika Tailoring Studio" is designed to pass. It has HTTPS with security headers, a short POST form with a purpose sentence, an unticked consent box, a privacy notice with a rights section, contact details and a Data Protection Commission statement. The demo buttons scan these from disk, so a demonstration needs no internet.

5. SECURITY CONSIDERATIONS

Because this is a security tool, it has to follow its own advice. Table 5.1 lists the main risks and how the code deals with them.

Risk	Protection in Accolm Check
SSRF: reaching localhost, the private network or cloud metadata	http/https only; ports 80, 443, 8080 and 8443; localhost, .local and .internal names refused; private, loopback, link-local, CGNAT, multicast and reserved IPv4 and IPv6 ranges blocked, including IPv4-mapped IPv6 and numeric forms such as http://2130706433/.
DNS rebinding	The IP is checked inside the socket's DNS lookup; if any resolved address is private, the request is refused.
Redirect to an internal address	At most 3 redirects, each validated again from scratch.
Slow or huge responses	8-second overall deadline and 1.5 MB size limit; at most 2 pages per scan.
Abuse of the scanner	Rate limit of 10 scans per minute per IP address; 4 KB limit on request bodies.
SQL injection	Prepared statements with ? placeholders everywhere; strict validation of ids.
Cross-site scripting from a hostile scanned page	All text from scanned sites is HTML-escaped before it is shown. The app's own Content-Security-Policy allows only its own scripts, with no inline scripts.
Clickjacking and other browser attacks on the tool	X-Frame-Options: DENY, CSP frame-ancestors 'none', X-Content-Type-Options: nosniff, Referrer-Policy: no-referrer, and HSTS when served over HTTPS.
Storing personal data	Only addresses, field categories and findings are stored, never values typed by people.

Table 5.1: Security risks and protections

One setting needs care. Some networks, such as certain VPNs and sandboxes, answer every DNS query with an address in the 198.18.0.0/15 range and pass the traffic on through a tunnel. The scanner blocks that range, so on such a network every live scan is refused. The optional TRUSTED_EGRESS_CIDRS setting allows only the range it names. It is empty by default, and loopback and private ranges stay blocked even when it is set. I needed it to test live scans in my development environment.

6. TESTING AND RESULTS

6.1 Automated tests

I wrote 27 automated tests with the test runner built into Node.js (node --test). They run with npm test and finish in about one second. All 27 passed (Table 6.1).

Test file	Tests	What is tested	Result
test/scanner.test.js	12	Both demo sites give the expected statuses; every finding maps to s.17(a) to (h); Ghana field recognition; no form penalties on pages without forms; GET and http form actions; pre-ticked consent; trackers with and without a cookie notice; security headers; Ghana phone and digital address; privacy links stay on the same site; hostile page content	12 passed
test/ssrf.test.js	10	18 blocked IP forms and 5 allowed public IPs; 20 unsafe URLs refused (file://, ftp://, localhost, 127.0.0.1, 2130706433, 0x7f.0.0.1, [::1], 169.254.169.254, user:password@, port 22 and others); DNS rebinding; a redirect to cloud metadata; the size limit; the timeout; TRUSTED_EGRESS_CIDRS cannot open loopback	10 passed
test/api.test.js	5	The API refuses internal targets with status 400; a demo scan is saved and read back; unknown demo names and SQL-injection-style ids are refused; the app's own security headers; rate limiting returns status 429	5 passed

Table 6.1: Automated test results (27 of 27 passed)

6.2 End-to-end results

I ran the app locally and scanned both demo sites and one live public site, example.com, a domain reserved for documentation (Table 6.2). The non-compliant demo failed 8 checks and got 5 warnings, scoring 14 ("At risk"). The compliant demo passed all 14 checks and scored 100 ("Strong start"). example.com has no forms, so the form checks did not apply. It scored 38 ("At risk") because it has no privacy notice, contact details or security headers. That is a fair result for a placeholder page, and it shows the score is about privacy habits, not about whether a site "works". The request to scan http://169.254.169.254/latest was refused with "Private, loopback and internal IP addresses cannot be scanned."

Site	Score	Band	Pass / check / fix now
Adwoa's Kitchen Catering (demo, non-compliant)	14	At risk	0 / 5 / 8
Sika Tailoring Studio (demo, compliant)	100	Strong start	14 / 0 / 0
example.com (live)	38	At risk	3 / 1 / 4

Table 6.2: End-to-end scan results

6.3 Screenshots

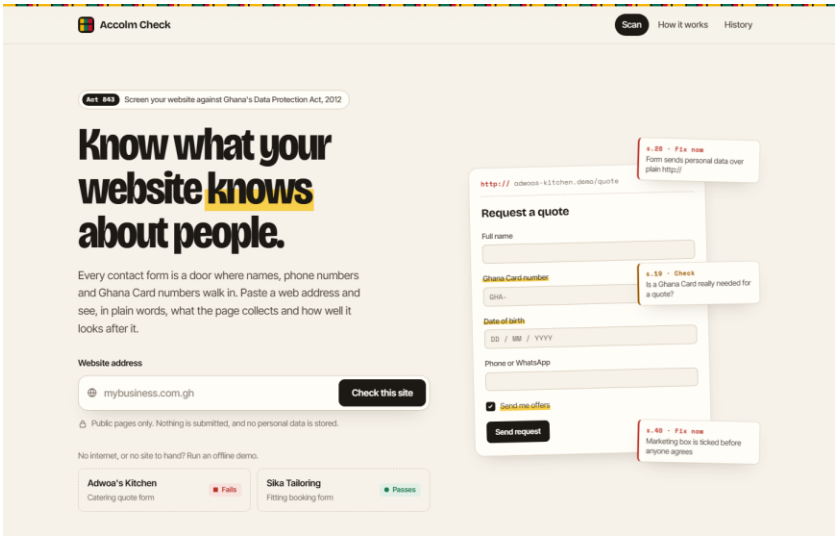


Figure 6.1: Home page with the scan form, demo buttons and a marked-up sample form

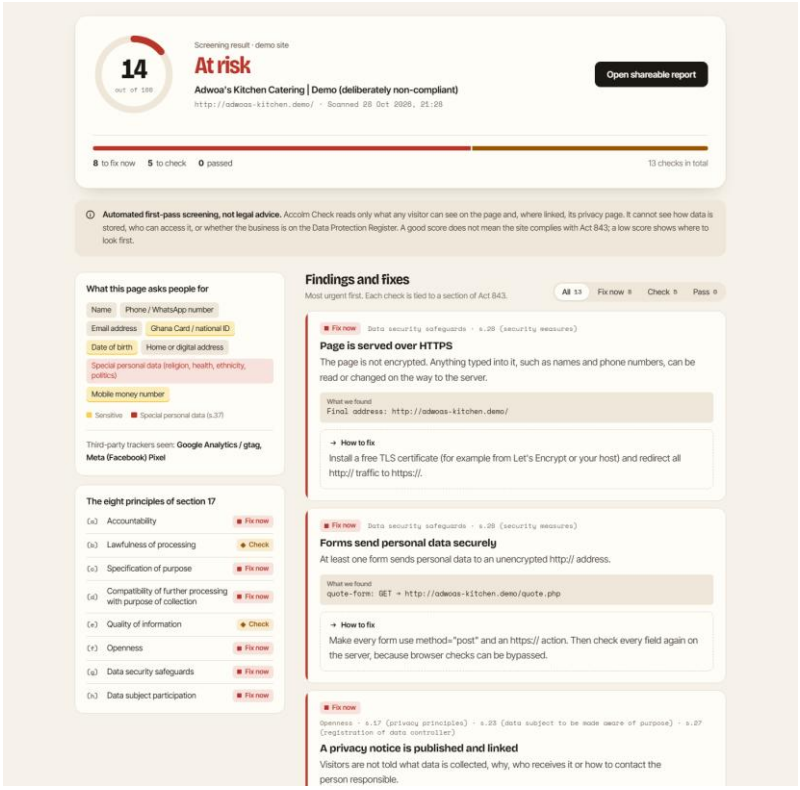


Figure 6.2: Scan result for the non-compliant demo: score ring, data inventory, the eight principles and the first findings

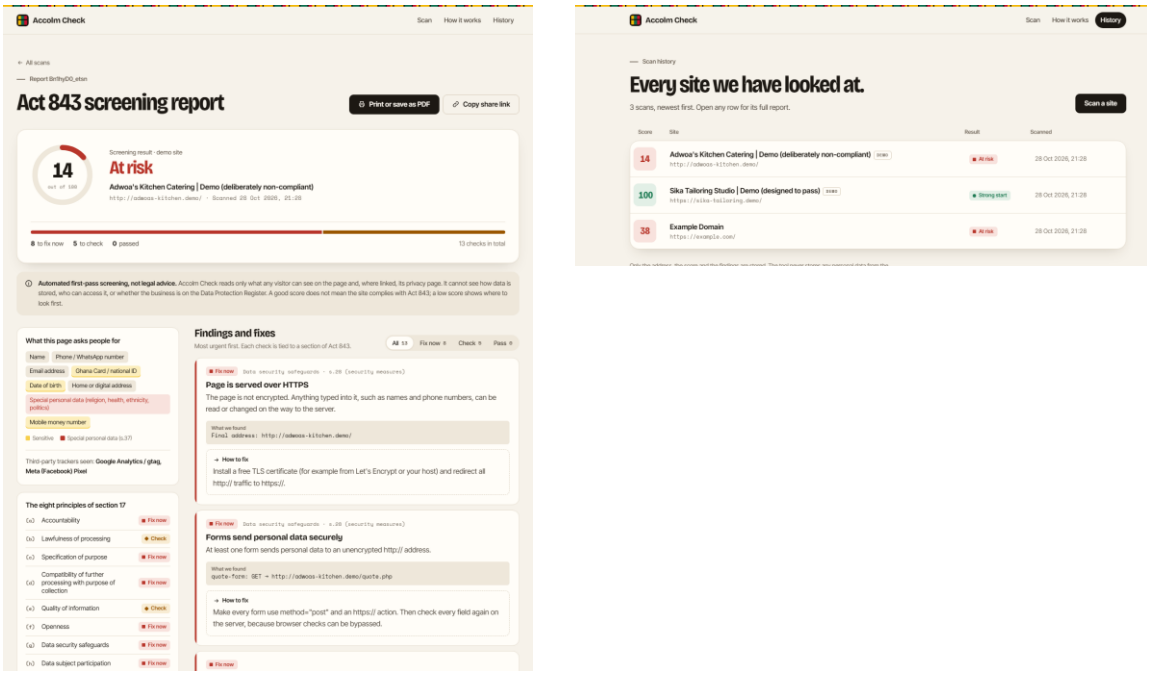


Figure 6.3: Shareable report page, also printable as an A4 PDF (left), and the scan history page (right)

7. LIMITATIONS, FUTURE WORK AND CONCLUSION

7.1 Limitations

- It reads only public HTML. It cannot see the server, the database, staff access or how long data is kept, which are central to the security safeguards and accountability principles.
- It does not run JavaScript. Forms or trackers that are added after the page loads, as in many React sites and website builders, may be missed.
- Field recognition uses keywords in English. A field labelled only in another language, or with a meaningless name such as field_7, may not be recognised.
- It cannot confirm registration with the Data Protection Commission. It only looks for a statement on the site.
- The weights and score bands are my own judgement, not an official standard, and the Act's other lawful reasons for processing (such as a contract) cannot be seen from a page.
- It scans one page and its privacy page, not a whole site.

7.2 Future work

- Use a headless browser for pages built with JavaScript.
- Crawl a few more pages of the same site, such as contact, booking and checkout pages.
- Add Twi and other Ghanaian language keywords, and French for regional clients.
- Produce a ready-made privacy notice template from the scan results, for the business to complete.
- Re-scan regularly and alert the owner by email or WhatsApp when a score drops.
- Ask the Data Protection Commission whether the public register can be checked automatically.

7.3 Conclusion

During my attachment with Accolm I learnt that a form is where personal data enters a system, and that browser checks are not security. Accolm Check puts those two lessons into a working tool. In seconds, it shows a business owner what their website asks people for, how well the page protects that data, and which of the eight principles in section 17 need attention, with a fix for each. The tool also follows its own advice, with server-side validation, prepared statements, SSRF protection and strict security headers, and 27 automated tests check its behaviour. It does not replace legal advice, but it gives small Ghanaian businesses, and web designers like me, a practical first step towards treating people's data with care.

[Confirm/insert: whether I discussed this project idea with my Accolm work-based supervisor or my academic supervisor, and any feedback received]

REFERENCES

Accolm. (n.d.-a). Accolm home page. Retrieved October 20, 2026, from https://www.accolm.com/site/

Accolm. (n.d.-h). Web application security. Retrieved October 20, 2026, from https://www.accolm.com/site/solutions/security/web-application-security

Bootstrap. (n.d.). Bootstrap documentation. Retrieved October 28, 2026, from https://getbootstrap.com/docs/5.3/

Cotton, M., Vegoda, L., Bonica, R., & Haberman, B. (2013). Special-purpose IP address registries (RFC 6890). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc6890

OpenJS Foundation. (n.d.). Express: Fast, unopinionated, minimalist web framework for Node.js. Retrieved October 28, 2026, from https://expressjs.com/

OWASP Foundation. (2021). OWASP Top 10: A10:2021 – Server-Side Request Forgery (SSRF). https://owasp.org/Top10/A10_2021-Server-Side_Request_Forgery_%28SSRF%29/

OWASP Foundation. (n.d.-a). Server Side Request Forgery prevention cheat sheet. Retrieved October 28, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html

OWASP Foundation. (n.d.-b). Input validation cheat sheet. Retrieved October 28, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

OWASP Foundation. (n.d.-c). SQL injection prevention cheat sheet. Retrieved October 28, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

OWASP Foundation. (n.d.-d). OWASP Secure Headers Project. Retrieved October 28, 2026, from https://owasp.org/www-project-secure-headers/

Republic of Ghana. (2012). Data Protection Act, 2012 (Act 843). GhaLII. https://ghalii.org/akn/gh/act/2012/843/eng@2012-05-18

Sommerville, I. (2016). Software engineering (10th ed.). Pearson.